Department of Computer Science & Engineering Education

Information Retrieval Techniques

NAME - RAJ

ENROLLMENT NO. - NITRB-25-BD-PG-010

COURSE - M.TECH CSE(BIG DATA ANALYTICS)

GithubLink
https://github.com/rajsanodiya122/Information_Retrieval_Techniques

Unit-3

Activity 5 : Smart Email Filtering System for Corporate Communication

Problem Statement

A multinational company receives thousands of emails daily across departments such as HR, Finance, Technical Support, and Marketing. Employees are facing issues because:

- Important emails are getting mixed with spam
- Emails are not categorized properly
- Manual sorting is inefficient

The IT department decides to build an Intelligent Email Filtering System that can:

- Automatically classify emails into categories such as:
  - Spam
  - Work/Official
- Improve system performance by reducing unnecessary features
- Evaluate model performance for reliability.

Objective

- Automate email categorization using NLP preprocessing and classification techniques.
- Convert raw text into TF-IDF vectors for mathematical feature extraction.
- Train machine learning models to accurately identify spam and official mail.
- Optimize system performance through dimensionality reduction and accuracy evaluation

Dataset

Example Dataset is as Fallows

text,label
Win a free iPhone click now,spam
Meeting scheduled tomorrow at 10 AM,ham
Limited time offer buy now,spam
Project update attached please review,ham

Real Dataset

Students can use: SMS Spam Dataset (Kaggle)/or use any public data set for given problem.

Dataset contains:

- Spam emails/messages
- Ham (official/work) emails/messages

Dataset Loading

[24]:
```
#TASK 1
```

[29]:
```
import pandas as pd
import nltk
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_selection import SelectKBest, chi2
```

```
[nltk_data] Downloading package punkt to /home/hduser/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /home/hduser/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /home/hduser/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /home/hduser/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

[30]:
```
df = pd.read_csv("combined_dataset.csv")

print("Original Dataset:")
print(df.head())

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
```

```
Original Dataset:
  target                                               text
0   spam  Congratulations! You've been selected for a lu...
1   spam  URGENT: Your account has been compromised. Cli...
2   spam  You've won a free iPhone! Claim your prize by ...
3   spam  Act now and receive a 50% discount on all purc...
4   spam  Important notice: Your subscription will expir...
```

Methodology

Text Preprocessing

The following preprocessing techniques were applied:

* Lowercasing
* Tokenization
* Stopword Removal
* Lemmatization

This helped clean the raw text data.

[31]:
```
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return " ".join(tokens)
```

[32]:
```
df['clean_text'] = df['text'].apply(preprocess_text)

print(df[['text', 'clean_text', 'target']].head())

df.to_csv("cleaned_emails.csv", index=False)
```

```
                                                text  \
0  Congratulations! You've been selected for a lu...
1  URGENT: Your account has been compromised. Cli...
2  You've won a free iPhone! Claim your prize by ...
3  Act now and receive a 50% discount on all purc...
4  Important notice: Your subscription will expir...

                                          clean_text target
0  congratulation youve selected luxury vacation ...   spam
1  urgent account compromised click reset passwor...   spam
2        youve free iphone claim prize clicking link   spam
3   act receive discount purchase limited time offer   spam
4  important notice subscription expire soon rene...   spam
```

Feature Extraction using TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency) was used to convert text into numerical vectors.

This assigns weights to important words and removes less relevant words.

Output:
Total TF-IDF features generated:

33]:
```
#TASK 2
df = pd.read_csv("cleaned_emails.csv")

print("Cleaned Dataset:")
print(df[['clean_text', 'target']].head())
```

```
Cleaned Dataset:
                                          clean_text target
0  congratulation youve selected luxury vacation ...   spam
1  urgent account compromised click reset passwor...   spam
2        youve free iphone claim prize clicking link   spam
3   act receive discount purchase limited time offer   spam
4  important notice subscription expire soon rene...   spam
```

34]:
```
# Initialize TF-IDF Vectorizer
tfidf = TfidfVectorizer()

# Handle potential NaN values in 'clean_text' by filling them with an empty string
df['clean_text'] = df['clean_text'].fillna('')

# Convert text into TF-IDF vectors
X = tfidf.fit_transform(df['clean_text'])
y = df['target']

# Display shape of TF-IDF matrix
print("\nTF-IDF Matrix Shape:")
print(X.shape)

# Display number of features
print("\nNumber of Features:")
print(len(tfidf.get_feature_names_out()))

# Display feature names (optional)
print("\nSample Features:")
print(tfidf.get_feature_names_out()[:20])
```

```
TF-IDF Matrix Shape:
(10961, 47419)

Number of Features:
47419

Sample Features:
['aa' 'aaa' 'aabda' 'aabvmmq' 'aac' 'aachecar' 'aaer' 'aafco' 'aah'
 'aaiabe' 'aaigrcrb' 'aaihmqv' 'aaldano' 'aalland' 'aambique' 'aamlrg'
 'aaniye' 'aaoeuro' 'aaoooooright' 'aare']
```

Model Training

```
35]: #TASK 3
df = pd.read_csv("cleaned_emails.csv")

df['clean_text'] = df['clean_text'].fillna('')

print("Columns in dataset:", df.columns)

# Features
X_text = df['clean_text']

y = df['target']

# TF-IDF

tfidf = TfidfVectorizer(max_features=5000)

X = tfidf.fit_transform(X_text)


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Columns in dataset: Index(['target', 'text', 'clean_text'], dtype='str')
Training data shape: (8768, 5000)
Testing data shape: (2193, 5000)
```

Two machine learning models were used:

A. Logistic Regression

* Trained using TF-IDF features
* Used for email classification

36]:
```
# LogisticRegression
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train, y_train)

# Predictions
lr_pred = lr_model.predict(X_test)

# Accuracy
lr_accuracy = accuracy_score(y_test, lr_pred)

print("Logistic Regression Accuracy:", lr_accuracy)

# Detailed report
print("\nClassification Report:")
print(classification_report(y_test, lr_pred))
```

```
Logistic Regression Accuracy: 0.9357045143638851

Classification Report:
              precision    recall  f1-score   support

         ham       0.93      0.99      0.96      1706
        spam       0.96      0.74      0.84       487

    accuracy                           0.94      2193
   macro avg       0.94      0.87      0.90      2193
weighted avg       0.94      0.94      0.93      2193
```

B. Naive Bayes

* Trained using TF-IDF features
* Faster model for text classification

[37]:
```
# Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)

nb_pred = nb_model.predict(X_test)

nb_accuracy = accuracy_score(y_test, nb_pred)

print("Naive Bayes Accuracy:", nb_accuracy)

print("\nClassification Report:")
print(classification_report(y_test, nb_pred))
```

```
Naive Bayes Accuracy: 0.9430004559963521

Classification Report:
              precision    recall  f1-score   support

         ham       0.95      0.98      0.96      1706
        spam       0.91      0.82      0.87       487

    accuracy                           0.94      2193
   macro avg       0.93      0.90      0.91      2193
weighted avg       0.94      0.94      0.94      2193
```

Feature Selection / Dimensionality Reduction

Chi-Square feature selection was applied to remove unnecessary features.

[38]:
```
#TASK 4
# First split original TF-IDF data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Apply feature selection only on training data
selector = SelectKBest(chi2, k=1000)

X_train_sel = selector.fit_transform(X_train, y_train)
X_test_sel = selector.transform(X_test)

print("Original Features:", X.shape[1])
print("Reduced Features:", X_train_sel.shape[1])
```

```
Original Features: 5000
Reduced Features: 1000
```

[39]:
```
X_train_sel, X_test_sel, y_train_sel, y_test_sel = train_test_split(
    X_selected, y, test_size=0.2, random_state=42
)
```

[40]:
```
lr_model2 = LogisticRegression(max_iter=1000)
lr_model2.fit(X_train_sel, y_train_sel)

pred_sel = lr_model2.predict(X_test_sel)

reduced_accuracy = accuracy_score(y_test_sel, pred_sel)

print("Accuracy After Feature Selection:", reduced_accuracy)
```

```
Accuracy After Feature Selection: 0.925672594619243
```

[41]:
```
#TASK 5
print("Accuracy Before Reduction:", lr_accuracy)
print("Accuracy After Reduction:", reduced_accuracy)
```

```
Accuracy Before Reduction: 0.9357045143638851
Accuracy After Reduction: 0.925672594619243
```

Observations

* Text preprocessing improved data quality
* TF-IDF successfully converted text into machine-readable format
* Logistic Regression achieved high accuracy
* Naive Bayes provided faster execution
* Feature selection reduced dimensionality
* Model performance remained strong after optimization

Conclusion

The Intelligent Email Filtering System successfully classified emails into spam and work categories using NLP and machine learning techniques.

The project automated email sorting, reduced manual effort, and improved efficiency. Feature selection further optimized system performance by reducing unnecessary features while maintaining classification accuracy.

This system can help organizations manage large volumes of emails more effectively.

Tools & Technologies Used

* Python
* Pandas
* NLTK
* Scikit-learn
* TF-IDF Vectorizer
* Logistic Regression
* Naive Bayes
* Chi-Square Feature Selection
* Jupyter Notebook

Outcome

✓ Cleaned raw email text

✓ Converted emails into TF-IDF vectors

✓ Classified emails accurately

✓ Improved performance using feature reduction